

10 — Query Optimization Cookbook

"Real-world optimization scenarios with before/after EXPLAIN comparisons."

Table of Contents

1. [How to Use This Cookbook](#)
 2. [Scenario 01: Missing Index on Foreign Key](#)
 3. [Scenario 02: Composite Index Column Order](#)
 4. [Scenario 03: Function Breaking Index Usage](#)
 5. [Scenario 04: N+1 Query Elimination](#)
 6. [Scenario 05: OFFSET Pagination at Scale](#)
 7. [Scenario 06: Sorting Without Index \(work mem Spill\)](#)
 8. [Scenario 07: Stale Statistics Causing Wrong Plan](#)
 9. [Scenario 08: COUNT\(*\) on Large Table](#)
 10. [Scenario 09: JSONB Field Lookup Without Index](#)
 11. [Scenario 10: Full-Text Search Setup](#)
 12. [Scenario 11: Slow DISTINCT Query](#)
 13. [Scenario 12: NOT IN with Possible NULL](#)
 14. [Scenario 13: Partial Index for Hot Subset](#)
 15. [Scenario 14: Covering Index Elimination](#)
 16. [Scenario 15: Aggregation With Large Sort](#)
 17. [Scenario 16: Parallel Query Enablement](#)
 18. [Scenario 17: CTE vs Subquery Performance](#)
 19. [Scenario 18: LIKE with Trigram Index](#)
 20. [Scenario 19: Correlated Subquery to JOIN](#)
 21. [Scenario 20: Multi-Tenant Index Design](#)
 22. [Scenario 21: BRIN for Time-Series Data](#)
 23. [Scenario 22: Hash Join Memory Optimization](#)
 24. [Scenario 23: Index-Only Scan Enablement](#)
 25. [Interview Questions & Answers](#)
 26. [Cross-References](#)
-

How to Use This Cookbook

Each scenario follows this structure:

1. **Setup** — table schema and data
 2. **Problem** — the slow query
 3. **BEFORE** — EXPLAIN ANALYZE showing the bad plan
 4. **Root Cause** — why it's slow
 5. **Fix** — the optimization
 6. **AFTER** — EXPLAIN ANALYZE showing the improvement
 7. **Key Insight** — the lesson to remember
-

Scenario 01: Missing Index on Foreign Key

Setup

```
CREATE TABLE customers (id BIGSERIAL PRIMARY KEY, name TEXT, email TEXT);
CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  customer_id BIGINT REFERENCES customers(id),
  status TEXT,
  total NUMERIC,
  created_at TIMESTAMPTZ DEFAULT now()
);
-- 10M orders, 100K customers
```

Problem

```
SELECT c.name, COUNT(o.id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
GROUP BY c.id, c.name;
```

BEFORE

```
Hash Left Join  (cost=34567.89..789012.34 rows=100000 width=20)
                  (actual time=567.234..45678.901 rows=100000 loops=1)
    Hash Cond: (c.id = o.customer_id)
    -> Seq Scan on customers  (rows=100000 actual=100000)
    -> Hash  (Batches: 8  Memory Usage: 4096kB)  ← DISK SPILL!
        -> Seq Scan on orders  (rows=10000000 actual=10000000)
            Buffers: shared hit=100 read=45000  ← 45K disk reads!
Execution Time: 45678 ms (45 seconds!)
```

Root Cause

- No index on `orders.customer_id` → full table seq scan of 10M rows
- Hash table batching (8 batches) due to `work_mem` issue
- 45K disk reads pulling all order pages

Fix

```
-- Add the missing FK index:
CREATE INDEX CONCURRENTLY idx_orders_customer_id ON orders(customer_id);
-- Also increase work_mem to eliminate batching:
SET work_mem = '128MB';
```

AFTER

```
Hash Left Join  (cost=5678.90..23456.78 rows=100000 width=20)
                  (actual time=45.678..2345.678 rows=100000 loops=1)
    Hash Cond: (c.id = o.customer_id)
    -> Seq Scan on customers  (rows=100000 actual=100000)
    -> Hash  (Batches: 1  Memory Usage: 12345kB)  ← fits in memory!
```

```
-> Index Scan using idx_orders_customer_id on orders
      (rows=100000000 actual=10000000)
      Buffers: shared hit=30000 read=100  - 100 reads vs 45000!
Execution Time: 2345 ms (2.3 seconds)
```

Key Insight

Always index foreign keys in PostgreSQL. The planner can't use an index that doesn't exist — create it with CONCURRENTLY to avoid locking.

Scenario 02: Composite Index Column Order

Setup

```
CREATE TABLE events (
  id BIGSERIAL PRIMARY KEY,
  tenant_id BIGINT,
  event_type TEXT,
  amount NUMERIC,
  created_at TIMESTAMPTZ DEFAULT now()
);
-- Existing index: CREATE INDEX ON events(event_type, tenant_id);
```

Problem

```
SELECT * FROM events
WHERE tenant_id = 42
      AND event_type = 'purchase'
      AND created_at > NOW() - INTERVAL '30 days';
```

BEFORE

```
Index Scan using idx_events_type_tenant on events
  (cost=0.56..45678.90 rows=500 width=64)
  (actual time=0.045..8900.123 rows=487 loops=1)
  Index Cond: ((event_type = 'purchase') AND (tenant_id = 42))
  Filter: (created_at > (now() - '30 days'::interval))
  Rows Removed by Filter: 49513  - 49K rows fetched and discarded!
  Buffers: shared hit=1234 read=5678
Execution Time: 8900 ms
```

Root Cause

- Index on (event_type, tenant_id) — correct columns but wrong order
- event_type has low cardinality (maybe 20 types): selectivity ~5%
- tenant_id has high cardinality (10K tenants): selectivity ~0.01%
- Should put the more selective column first, AND add created_at for the date filter

Fix

```
-- Drop old index, create better one with high-cardinality column first + date
DROP INDEX CONCURRENTLY idx_events_type_tenant;
CREATE INDEX CONCURRENTLY idx_events_tenant_type_date
ON events(tenant_id, event_type, created_at DESC);
```

AFTER

```
Index Scan using idx_events_tenant_type_date on events
(cost=0.56..234.56 rows=490 width=64)
(actual time=0.045..1.234 rows=487 loops=1)
Index Cond: ((tenant_id = 42) AND (event_type = 'purchase')
            AND (created_at > ...))
Buffers: shared hit=12
Execution Time: 1.5 ms
```

Key Insight

Equality columns before range columns. Most selective equality column first. Add the date range column at the end to enable efficient time-range filtering without heap access.

Scenario 03: Function Breaking Index Usage

Setup

```
CREATE TABLE users (id BIGSERIAL PRIMARY KEY, email TEXT, name TEXT);
CREATE INDEX idx_users_email ON users(email);
-- 5M users
```

Problem

```
SELECT * FROM users WHERE lower(email) = lower($1);
```

BEFORE

```
Seq Scan on users (cost=0.00..98765.00 rows=5 width=64)
(actual time=0.023..12345.678 rows=1 loops=1)
Filter: (lower(email) = lower($1))
Rows Removed by Filter: 4999999
Buffers: shared hit=100 read=44444
Execution Time: 12345 ms (12 seconds for 1 row!)
```

Root Cause

`lower(email)` is applied to the `email` column, preventing the index on raw `email` from being used. The entire table is scanned and `lower()` computed for every row.

Fix

```
-- Create expression index matching the function in the query:
CREATE INDEX CONCURRENTLY idx_users_lower_email ON users(lower(email));
-- Optional: also make it unique (prevents duplicate case-insensitive emails):
CREATE UNIQUE INDEX CONCURRENTLY idx_users_lower_email_unique ON
users(lower(email));
```

AFTER

```
Index Scan using idx_users_lower_email_unique on users
(cost=0.56..8.58 rows=1 width=64)
(actual time=0.034..0.034 rows=1 loops=1)
Index Cond: (lower(email) = lower($1))
Buffers: shared hit=4
Execution Time: 0.056 ms
```

Key Insight

The index expression must match the query expression exactly. `lower(email)` in the index matches `lower(email)` in the WHERE clause. No function applied to an indexed column without a matching expression index.

Scenario 04: N+1 Query Elimination

Setup

```
CREATE TABLE products (id BIGSERIAL PRIMARY KEY, name TEXT, category_id BIGINT);
CREATE TABLE categories (id BIGSERIAL PRIMARY KEY, name TEXT, parent_id BIGINT);
-- 100K products, 500 categories
```

Problem

Application code:

```
products = db.execute("SELECT id, name, category_id FROM products WHERE featured =
true")
for p in products: # 1000 featured products
    category = db.execute("SELECT name FROM categories WHERE id = %s",
p.category_id)
    print(f"{p.name} in {category.name}")
# 1001 queries instead of 1!
```

BEFORE (application-level)

```
Query 1: SELECT id, name, category_id FROM products WHERE featured = true
→ 1000 rows
```

```
Query 2..1001: SELECT name FROM categories WHERE id = $1
```

→ Each takes $\sim 0.5\text{ms} \times 1000 = 500\text{ms}$ overhead + network RTT
Total: 1-5 seconds depending on network latency

Fix

```
-- Replace 1001 queries with 1 JOIN:
SELECT p.id, p.name, c.name AS category_name
FROM products p
JOIN categories c ON p.category_id = c.id
WHERE p.featured = true;
```

AFTER

```
Hash Join (cost=12.34..456.78 rows=1000 width=40)
    (actual time=1.234..5.678 rows=1000 loops=1)
    Hash Cond: (p.category_id = c.id)
    -> Index Scan using idx_products_featured on products
        (rows=1000 actual=1000)
        Buffers: shared hit=12
    -> Hash (Batches: 1 Memory Usage: 45kB)
        -> Seq Scan on categories (rows=500 actual=500)
            Buffers: shared hit=5
Execution Time: 6 ms (vs 1000+ ms before)
```

Key Insight

Every N+1 pattern can be replaced with a single JOIN. The round-trip overhead of N individual queries always dominates the actual query execution time.

Scenario 05: OFFSET Pagination at Scale

Setup

```
CREATE TABLE articles (
    id BIGSERIAL PRIMARY KEY,
    published_at TIMESTAMPTZ DEFAULT now(),
    title TEXT,
    author_id BIGINT
);
CREATE INDEX ON articles(published_at DESC, id DESC);
-- 10M articles
```

Problem

```
-- Page 5000: OFFSET 99980 LIMIT 20
SELECT id, title, published_at FROM articles
ORDER BY published_at DESC LIMIT 20 OFFSET 99980;
```

BEFORE

```
Limit  (cost=123456.78..123456.98 rows=20 width=40)
      (actual time=4567.890..4567.912 rows=20 loops=1)
    -> Index Scan using idx_articles_published on articles
          (cost=0.56..6789012.34 rows=10000000 width=40)
          (actual time=0.034..4567.456 rows=100000 loops=1)
Execution Time: 4568 ms  ← gets slower with every page!
```

Root Cause

OFFSET forces reading and discarding 99,980 rows even though we only want 20. Time grows linearly with page number.

Fix

```
-- Keyset pagination: use cursor from previous page's last row
-- (last row from page 4999: published_at='2024-01-01 00:00:00', id=543210)
SELECT id, title, published_at FROM articles
WHERE (published_at, id) < ('2024-01-01 00:00:00', 543210)
ORDER BY published_at DESC, id DESC
LIMIT 20;
```

AFTER

```
Limit  (cost=0.56..24.56 rows=20 width=40)
      (actual time=0.034..0.089 rows=20 loops=1)
    -> Index Scan using idx_articles_published on articles
          (cost=0.56..12000000.00 rows=...)
          (actual time=0.034..0.067 rows=20 loops=1)
          Index Cond: ((published_at, id) < ('2024-01-01', 543210))
          Buffers: shared hit=5
Execution Time: 0.12 ms  ← constant time regardless of page!
```

Key Insight

Keyset pagination is $O(\log N)$ regardless of page number. OFFSET pagination is $O(N \times \text{page_number})$.

Always implement cursor-based pagination for production systems.

Scenario 06: Sorting Without Index (work_mem Spill)

Setup

```
CREATE TABLE audit_log (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT,
  action TEXT,
  created_at TIMESTAMPTZ DEFAULT now())
```

```
);  
-- 50M rows, no composite index
```

Problem

```
SELECT user_id, action, created_at  
FROM audit_log  
WHERE user_id = 12345  
ORDER BY created_at DESC  
LIMIT 10;
```

BEFORE

```
Limit (cost=98765.43..98765.45 rows=10 width=24)  
  (actual time=23456.789..23456.792 rows=10 loops=1)  
    -> Sort (cost=98765.43..98815.43 rows=20000 width=24)  
          (actual time=23456.787..23456.789 rows=10 loops=1)  
          Sort Key: created_at DESC  
          Sort Method: external merge  Disk: 4096kB ← disk sort!  
          -> Index Scan using idx_audit_user_id on audit_log  
              (rows=20000 actual=19847)  
              Buffers: shared hit=2345, temp read=512 written=512  
Execution Time: 23457 ms (23 seconds for TOP 10!)
```

Root Cause

- Index on `user_id` alone: finds 20K rows for user 12345
- No sort order in index → must sort 20K rows
- Sort overflows `work_mem` → disk sort

Fix

```
-- Composite index with sort direction:  
CREATE INDEX CONCURRENTLY idx_audit_user_date  
ON audit_log(user_id, created_at DESC);
```

AFTER

```
Limit (cost=0.56..24.56 rows=10 width=24)  
  (actual time=0.045..0.089 rows=10 loops=1)  
    -> Index Scan using idx_audit_user_date on audit_log  
        (cost=0.56..49235.67 rows=20000 width=24)  
        (actual time=0.044..0.067 rows=10 loops=1)  
        Index Cond: (user_id = 12345)  
        Buffers: shared hit=5  
Execution Time: 0.12 ms ← 23 seconds → 0.1 ms!
```

Key Insight

A composite index with the correct sort direction eliminates both the heap scan and the sort operation.
With LIMIT, the planner stops after 10 rows — reading only 5 index pages.

Scenario 07: Stale Statistics Causing Wrong Plan

Setup

```
CREATE TABLE sessions (id BIGSERIAL PRIMARY KEY, user_id BIGINT, data JSONB,  
                        created_at TIMESTAMPTZ);  
-- Originally 100K rows; bulk loaded 50M more rows, forgot to ANALYZE
```

Problem

```
SELECT * FROM sessions WHERE user_id = 42 AND created_at > '2024-01-01';
```

BEFORE

```
Nested Loop  (cost=0.56..456.78 rows=2 width=512)  
    (actual time=0.034..456789.123 rows=5000 loops=1)  ← 5000 vs 2!  
  -> Index Scan using idx_sessions_user on sessions  
      (rows=2 actual=5000)  ← planner thinks 2, gets 5000!  
    ...  
Execution Time: 456789 ms (7+ MINUTES!)
```

Root Cause

- Statistics were for 100K rows (before bulk load of 50M)
- Planner estimated 2 rows, actual was 5000
- Chose Nested Loop (correct for 2 rows, catastrophic for 5000)

Fix

```
VACUUM ANALYZE sessions;  
-- or just:  
ANALYZE sessions;
```

AFTER

```
Hash Join  (cost=...)   
    (actual time=45.678..1234.567 rows=5000 loops=1)  
  -- Correct plan now that statistics are accurate  
Execution Time: 1234 ms (7 minutes → 1.2 seconds)
```

Key Insight

Always run ANALYZE after bulk data loads. Statistics are the planner's only window into your data — stale statistics = wrong plans = terrible performance.

Scenario 08: COUNT(*) on Large Table

Setup

```
-- 100M row orders table
-- Existing index: orders_pkey (id bigserial)
```

Problem

```
SELECT COUNT(*) FROM orders;
-- Takes 45 seconds with seq scan!
```

BEFORE

```
Aggregate  (cost=...)
-> Seq Scan on orders  (rows=100000000 actual=100000000)
    Buffers: shared hit=1000 read=499000  ← 499K disk reads!
Execution Time: 45234 ms
```

Root Cause

No index to use for COUNT(*). Must read entire table.

Fix Option 1: Use a narrow index

```
-- Create a tiny index on a NOT NULL column (status, for example):
CREATE INDEX CONCURRENTLY idx_orders_count ON orders(status) WHERE status IS NOT
NULL;
-- Or use the existing PK:
-- SELECT COUNT(*) can use any index that covers all rows
```

Fix Option 2: Approximate count (very fast)

```
-- For approximate count (within ~5% for most tables):
SELECT reltuples::BIGINT AS approx_count
FROM pg_class
WHERE relname = 'orders';
-- Returns instantly! Updated by VACUUM/ANALYZE
```

Fix Option 3: Materialized counter

```
-- If exact count needed frequently: maintain a counter table
CREATE TABLE table_counts (table_name TEXT PRIMARY KEY, row_count BIGINT);
INSERT INTO table_counts VALUES ('orders', 0);
-- Increment via trigger on INSERT/DELETE
```

AFTER (parallel seq scan)

```
Finalize Aggregate
-> Gather (Workers Planned: 8)
    -> Partial Aggregate
        -> Parallel Seq Scan on orders
Execution Time: 6789 ms ← 45s → 7s with parallelism
```

Key Insight

COUNT(*) on large tables is inherently expensive without tricks. Use approximate counts from `pg_class.reltuples` for monitoring, or maintain a counter for frequently-needed exact counts.

Scenario 09: JSONB Field Lookup Without Index

Setup

```
CREATE TABLE events (id BIGSERIAL PRIMARY KEY, payload JSONB, created_at
TIMESTAMPTZ);
-- 10M rows
```

Problem

```
SELECT * FROM events WHERE payload->>'event_type' = 'purchase';
```

BEFORE

```
Seq Scan on events (cost=0.00..456789.00 rows=100000 width=256)
    (actual time=0.034..23456.789 rows=98765 loops=1)
  Filter: ((payload ->> 'event_type') = 'purchase')
  Rows Removed by Filter: 9901235
  Buffers: shared hit=500 read=44444
Execution Time: 23457 ms
```

Root Cause

No index on the JSONB field `event_type`. Full table scan with JSONB extraction per row.

Fix

```
-- Expression index on the specific field:
CREATE INDEX CONCURRENTLY idx_events_event_type
ON events((payload->>'event_type'));

-- Or: GIN index for flexible multi-field queries:
CREATE INDEX CONCURRENTLY idx_events_payload
ON events USING GIN (payload jsonb_path_ops);
```

AFTER (expression index)

```
Bitmap Heap Scan on events (actual time=234.567..4567.890 rows=98765 loops=1)
  Recheck Cond: ((payload ->> 'event_type') = 'purchase')
  Heap Blocks: exact=44444
-> Bitmap Index Scan on idx_events_event_type
    (actual time=189.234..189.234 rows=98765 loops=1)
    Index Cond: ((payload ->> 'event_type') = 'purchase')
Execution Time: 4568 ms  ← 23s → 4.5s
```

Key Insight

For frequently-queried specific JSONB fields, an expression index is more efficient than a full GIN index.
Use GIN for flexible multi-field queries; use expression indexes for specific field lookups.

Scenario 10: Full-Text Search Setup

Setup

```
CREATE TABLE articles (id BIGSERIAL PRIMARY KEY, title TEXT, body TEXT,
                        published_at TIMESTAMPTZ);
-- 1M articles
```

Problem

```
SELECT * FROM articles
WHERE title ILIKE '%postgresql performance%'
      OR body ILIKE '%postgresql performance%';
-- Cannot use any index → seq scan, 60 seconds!
```

Fix

```
-- Step 1: Add tsvector column (pre-computed)
ALTER TABLE articles ADD COLUMN search_vec TSVECTOR
  GENERATED ALWAYS AS (
    to_tsvector('english', COALESCE(title, '') || ' ' || COALESCE(body, ''))
  ) STORED;

-- Step 2: GIN index
CREATE INDEX CONCURRENTLY idx_articles_fts ON articles USING GIN (search_vec);

-- Step 3: Rewrite query
SELECT id, title,
       ts_rank(search_vec, query) AS rank
FROM articles,
     plainto_tsquery('english', 'postgresql performance') AS query
WHERE search_vec @@ query
```

```
ORDER BY rank DESC
LIMIT 10;
```

AFTER

```
Limit
-> Sort (Sort Key: rank DESC; top-N heapsort)
    -> Bitmap Heap Scan on articles
        -> Bitmap Index Scan on idx_articles_fts
            Index Cond: (search_vec @@ plainto_tsquery('english', 'postgresql
performance'))
                (actual time=23.456..23.456 rows=4567 loops=1)
Execution Time: 234 ms ← 60 seconds → 234ms!
```

Key Insight

Full-text search requires `tsvector` + GIN index. Pre-compute the tsvector in a generated column (or via trigger) to avoid recomputing it on every query.

Scenario 11: Slow DISTINCT Query

Setup

```
-- orders table with 10M rows
-- customer_id column: 100K distinct customers
```

Problem

```
-- Find all customers who have placed at least one order
SELECT DISTINCT customer_id FROM orders;
```

BEFORE

```
HashAggregate (cost=123456.78..124456.78 rows=100000 width=8)
    (actual time=12345.678..12890.123 rows=100000 loops=1)
    Group Key: customer_id
    Batches: 4 Memory Usage: 8192kB ← disk spill!
    -> Seq Scan on orders (rows=10000000)
Execution Time: 12890 ms
```

Root Cause

DISTINCT on 10M rows with 100K distinct values — must scan everything, then deduplicate.

Fix

```

-- Better approach: EXISTS instead of DISTINCT
-- (avoids scanning all orders, uses index)
SELECT DISTINCT id FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

-- Or: use GIN/index directly
SELECT customer_id FROM orders
GROUP BY customer_id; -- similar performance to DISTINCT

-- If you need customer details too:
SELECT c.* FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

```

AFTER (EXISTS approach)

```

Hash Semi Join  (cost=12345.67..34567.89 rows=100000 width=8)
                (actual time=890.123..1234.567 rows=100000 loops=1)
  Hash Cond: (c.id = o.customer_id)
    -> Seq Scan on customers  (rows=100000)
    -> Hash  (rows=10000000)
          -> Index Only Scan using idx_orders_customer_id on orders
                (rows=10000000)
          Heap Fetches: 0
Execution Time: 1234 ms  ← 12s → 1.2s

```

Key Insight

Use **EXISTS** to check for "at least one matching row" — it stops at the first match. **DISTINCT** scans the entire result and then deduplicates.

Scenario 12: NOT IN with Possible NULL

Setup

```

CREATE TABLE blocked_ips (ip INET, reason TEXT);
-- Some rows have ip = NULL!

```

Problem

```

-- Find orders from non-blocked IPs
SELECT * FROM orders
WHERE client_ip NOT IN (SELECT ip FROM blocked_ips);
-- Returns ZERO rows when blocked_ips has any NULL!

```

Root Cause

SQL three-valued logic: `x NOT IN (... , NULL)` → `x != NULL` → UNKNOWN → treated as FALSE.

Fix

```
-- Option 1: NOT EXISTS (handles NULLs correctly)
SELECT * FROM orders o
WHERE NOT EXISTS (
    SELECT 1 FROM blocked_ips b
    WHERE b.ip = o.client_ip
    AND b.ip IS NOT NULL
);

-- Option 2: Exclude NULLs from subquery
SELECT * FROM orders
WHERE client_ip NOT IN (SELECT ip FROM blocked_ips WHERE ip IS NOT NULL);

-- Option 3: LEFT JOIN anti-join
SELECT o.*
FROM orders o
LEFT JOIN blocked_ips b ON o.client_ip = b.ip
WHERE b.ip IS NULL;
```

Key Insight

Never use NOT IN with a subquery that might return NULLs. Always use NOT EXISTS or add `IS NOT NULL` to the subquery.

Scenario 13: Partial Index for Hot Subset

Setup

```
CREATE TABLE jobs (
    id BIGSERIAL PRIMARY KEY,
    status TEXT, -- 'queued', 'running', 'done', 'failed'
    priority INT,
    created_at TIMESTAMPTZ DEFAULT now()
);
-- 100M rows: 99.9% 'done', 0.1% 'queued'/'running'
CREATE INDEX idx_jobs_status ON jobs(status, priority);
-- Full index: 1.6 GB (mostly useless 'done' entries)
```

Problem

```
-- Queue worker query (runs every 100ms):
SELECT * FROM jobs WHERE status = 'queued'
ORDER BY priority DESC, created_at LIMIT 10 FOR UPDATE SKIP LOCKED;
-- Full index (1.6 GB) not cached → cache thrashing!
```

Fix

```
-- Drop full index, create partial index for only active jobs:
DROP INDEX CONCURRENTLY idx_jobs_status;
CREATE INDEX CONCURRENTLY idx_jobs_active
ON jobs(priority DESC, created_at)
WHERE status IN ('queued', 'running');
-- Index size: 0.1% of 100M rows × ~20 bytes = ~20 MB vs 1.6 GB!
```

AFTER

```
Index Scan using idx_jobs_active on jobs
(actual time=0.023..0.067 rows=10 loops=1)
Index Cond: ((priority DESC, created_at) ORDER)
Filter: (status = 'queued')
Buffers: shared hit=3 ← 20 MB index fits entirely in cache
Execution Time: 0.1 ms ← 80× speedup!
```

Key Insight

Partial indexes are exponentially better when the predicate is highly selective. A 0.1% partial index is 1000× smaller than a full index, fitting entirely in cache.

Scenario 14: Covering Index Elimination

Setup

```
CREATE TABLE sessions (
  id BIGSERIAL PRIMARY KEY,
  token TEXT,
  user_id BIGINT,
  expires_at TIMESTAMPTZ
);
CREATE INDEX idx_sessions_token ON sessions(token);
-- Hot path: token validation on every HTTP request
```

Problem

```
SELECT user_id, expires_at FROM sessions WHERE token = $1;
-- High call rate (10K/sec): index scan + heap fetch every call
```

Fix

```
-- Convert to covering index:
DROP INDEX CONCURRENTLY idx_sessions_token;
CREATE INDEX CONCURRENTLY idx_sessions_token_cov
ON sessions(token) INCLUDE (user_id, expires_at);
VACUUM sessions; -- set all-visible bits
```


AFTER

```
Index Only Scan using idx_sessions_token_cov on sessions
(actual time=0.023..0.023 rows=1 loops=1)
Index Cond: (token = $1)
Heap Fetches: 0 ← zero heap reads!
Buffers: shared hit=3
Execution Time: 0.04 ms vs 0.12 ms before (3x speedup at 10K/sec = significant!)
```

Key Insight

Covering indexes eliminate heap fetches for the hottest queries. At 10,000 requests/sec, saving 2 heap page reads per request eliminates 20,000 random reads/second from shared_buffers.

Scenario 15: Aggregation With Large Sort

Setup

```
-- 50M order_items rows
```

Problem

```
SELECT product_id, SUM(quantity * unit_price) AS revenue
FROM order_items
GROUP BY product_id
ORDER BY revenue DESC
LIMIT 10;
```

BEFORE

```
Limit
-> Sort (Sort Method: external merge Disk: 256000kB) ← 256 MB disk sort!
    -> HashAggregate (Batches: 4 Memory Usage: 8192kB)
        -> Seq Scan on order_items (rows=50000000)
Execution Time: 456789 ms (7+ minutes)
```

Fix

```
-- Option 1: Increase work_mem for this session
SET work_mem = '512MB';

-- Option 2: Pre-aggregate with materialized view
CREATE MATERIALIZED VIEW product_revenue AS
SELECT product_id, SUM(quantity * unit_price) AS revenue
FROM order_items
GROUP BY product_id;
CREATE INDEX ON product_revenue(revenue DESC);
```

```
-- Refresh periodically:  
REFRESH MATERIALIZED VIEW CONCURRENTLY product_revenue;
```

AFTER (with work_mem)

```
Limit  
-> Sort (Sort Method: top-N heapsort Memory: 28kB) ← in-memory top-N!  
    -> HashAggregate (Batches: 1 Memory Usage: 204800kB) ← fits!  
        -> Seq Scan on order_items  
Execution Time: 12345 ms ← 7 minutes → 12 seconds
```

AFTER (materialized view)

```
Limit  
-> Index Scan using idx_product_revenue_desc on product_revenue  
    (rows=10 actual=10)  
Execution Time: 0.5 ms ← for repeated queries!
```

Key Insight

For repeated aggregate queries on large tables, materialized views are transformative. Pre-compute once, query instantly.

Scenario 16: Parallel Query Enablement

Setup

```
-- 1B row analytics table  
-- Server: 16-core CPU, 64 GB RAM  
-- Default: max_parallel_workers_per_gather = 2
```

Problem

```
SELECT product_category, SUM(revenue) FROM sales  
WHERE sale_date >= '2024-01-01'  
GROUP BY product_category;  
-- Takes 8 minutes with 2 parallel workers
```

Fix

```
-- Session-level parallel boost:  
SET max_parallel_workers_per_gather = 12;  
SET parallel_setup_cost = 100;  
  
-- System-level (postgresql.conf):  
ALTER SYSTEM SET max_parallel_workers_per_gather = 8;
```

```
ALTER SYSTEM SET max_parallel_workers = 32;
SELECT pg_reload_conf();
```

AFTER

```
Finalize HashAggregate
-> Gather (Workers Planned: 12 Workers Launched: 12)
    -> Partial HashAggregate
        -> Parallel Seq Scan on sales
            (loops=12 rows=83333333)
Execution Time: 45000 ms ← 8 minutes → 45 seconds (10x speedup)
```

Key Insight

On multi-core systems with OLAP workloads, increasing `max_parallel_workers_per_gather` to 8-12 provides near-linear speedup for large sequential scans.

Scenario 17: CTE vs Subquery Performance

Setup

PostgreSQL 12+ inlines CTEs, but understanding the difference matters for older versions and explicit `MATERIALIZED`.

Problem (pre-PG12 behavior)

```
WITH active_users AS (
    SELECT id FROM users WHERE is_active = true
)
SELECT u.id, COUNT(o.id) AS order_count
FROM active_users u
LEFT JOIN orders o ON u.id = o.user_id
GROUP BY u.id;
-- Pre-PG12: CTE is materialized, index on users.is_active not pushed through!
```

Fix (PG12+ behavior)

```
-- In PG12+: CTEs are automatically inlined (no optimization fence)
-- Above query is fine in PG12+

-- For explicit materialization (run CTE once, reuse result):
WITH expensive_data AS MATERIALIZED (
    SELECT user_id, COUNT(*) AS event_count
    FROM events WHERE created_at > NOW() - INTERVAL '30 days'
    GROUP BY user_id
)
SELECT u.name, ed.event_count
FROM users u
JOIN expensive_data ed ON u.id = ed.user_id
```

```
WHERE ed.event_count > 100;
-- MATERIALIZED: computed once, prevents re-execution if used multiple times in
query
```

Key Insight

In PostgreSQL 12+, CTEs are inlined by default (no optimization fence). Use `AS MATERIALIZED` only when the CTE result is expensive and reused multiple times, or when you explicitly want the fence behavior.

Scenario 18: LIKE with Trigram Index

Setup

```
CREATE TABLE products (id BIGSERIAL PRIMARY KEY, name TEXT, description TEXT);
-- 5M products
```

Problem

```
SELECT * FROM products WHERE name ILIKE '%samsung%';
-- Seq scan on 5M rows: 10 seconds
```

Fix

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX CONCURRENTLY idx_products_name_trgm
ON products USING GIN (name gin_trgm_ops);
```

AFTER

```
Bitmap Heap Scan on products
  Recheck Cond: ((name)::text ~~* '%samsung% '::text)
   -> Bitmap Index Scan on idx_products_name_trgm
        Index Cond: ((name)::text ~~* '%samsung% '::text)
        (actual time=45.678..45.678 rows=1234 loops=1)
Execution Time: 234 ms ← 10 seconds → 234ms
```

Key Insight

`pg_trgm` + GIN index enables $O(\log N)$ LIKE '%middle%' and ILIKE queries that standard B-tree cannot support.

Scenario 19: Correlated Subquery to JOIN

Problem

```
-- BAD: correlated subquery, executes once per order
SELECT o.id, o.total,
       (SELECT COUNT(*) FROM order_items oi WHERE oi.order_id = o.id) AS item_count
FROM orders o WHERE o.status = 'completed';
```

BEFORE

```
Seq Scan on orders (rows=8000000 completed)
  SubPlan:
    -> Aggregate (rows=1)
          -> Index Scan on order_items (loops=8000000) ← 8M subquery executions!
Execution Time: very slow
```

Fix

```
SELECT o.id, o.total, COUNT(oi.id) AS item_count
FROM orders o
LEFT JOIN order_items oi ON o.id = oi.order_id
WHERE o.status = 'completed'
GROUP BY o.id, o.total;
```

AFTER

```
HashAggregate
  -> Hash Left Join
        -> Index Scan on orders (status='completed')
        -> Hash (order_items)
Execution Time: fast single pass
```

Key Insight

Always replace correlated subqueries in SELECT lists with JOINS. The planner often can't optimize correlated subqueries as well as explicit joins.

Scenario 20: Multi-Tenant Index Design

Problem

A multi-tenant application was slow for all queries — no `tenant_id` in indexes.

Fix

```
-- Add tenant_id as leading column to ALL indexes:
CREATE INDEX CONCURRENTLY idx_orders_tenant_status ON orders(tenant_id, status,
created_at DESC);
CREATE INDEX CONCURRENTLY idx_users_tenant_email ON users(tenant_id, lower(email));

-- Ensure every query includes tenant_id:
```

```
SELECT * FROM orders WHERE tenant_id = $1 AND status = 'pending';
-- Uses idx_orders_tenant_status efficiently
```

Key Insight

In multi-tenant systems, `tenant_id` must be the leading column of every composite index. Without it, every query scans all tenants' data.

Scenario 21: BRIN for Time-Series Data

Setup

```
-- IoT sensor table: 10B rows, inserted in timestamp order
```

Problem

```
SELECT AVG(value) FROM sensor_readings WHERE recorded_at BETWEEN '2024-06-01' AND
'2024-06-30';
-- B-tree on recorded_at: 150 GB index, takes 5 minutes!
```

Fix

```
CREATE INDEX CONCURRENTLY idx_sensor_brin
ON sensor_readings USING BRIN (recorded_at)
WITH (autosummarize = on, pages_per_range = 64);
-- Index size: ~25 MB vs 150 GB for B-tree!
```

AFTER

```
Bitmap Heap Scan on sensor_readings
  Recheck Cond: (...)
  Heap Blocks: lossy=8000 ← only 8000 block ranges scanned vs 1.2M total!
    -> Bitmap Index Scan on idx_sensor_brin
Execution Time: 15000 ms ← 5 minutes → 15 seconds
```

Key Insight

For append-only time-series tables with billions of rows, BRIN indexes are 5,000× smaller than B-tree and provide excellent range query performance.

Scenario 22: Hash Join Memory Optimization

Problem

```
-- Large join timing out, pg_stat_statements shows temp_blks_written = 999999
```

```
SELECT * FROM large_table l JOIN medium_table m ON l.ref_id = m.id;
```

Hash Join

-> Seq Scan on large_table

-> Hash (Batches: 128 Memory Usage: 4096kB) ← 128 batches = disaster!

Execution Time: 987654 ms

Fix

```
-- For this query session:  
SET LOCAL work_mem = '1GB';  
-- Or: find the minimum needed:  
-- medium_table_rows × avg_width × 2.5 = required work_mem  
-- 1M rows × 100 bytes × 2.5 = 250 MB  
SET LOCAL work_mem = '256MB';
```

AFTER

Hash Join

-> Seq Scan on large_table

-> Hash (Batches: 1 Memory Usage: 256000kB) ← single batch!

Execution Time: 12345 ms ← 16 minutes → 12 seconds

Key Insight

Every Batches: N > 1 in a Hash node means the query needs more work_mem. Set `work_mem` per-session for known expensive queries.

Scenario 23: Index-Only Scan Enablement

Problem

Hot-path API: 50,000 requests/second, each running:

```
SELECT role, permissions FROM users WHERE id = $1;  
-- Index scan + heap fetch = 2 page reads × 50K/sec = 100K random reads/sec
```

Fix

```
-- Create covering index:  
CREATE INDEX CONCURRENTLY idx_users_id_cov ON users(id) INCLUDE (role, permissions);  
-- Run VACUUM to set visibility map:  
VACUUM users;
```

AFTER

```
Index Only Scan using idx_users_id_cov on users
Heap Fetches: 0 → zero heap reads!
(actual time=0.015..0.015 rows=1 loops=1)
-- 100K heap reads/sec → 0
```

Key Insight

Covering indexes can reduce I/O by 50-66% for lookup-heavy API endpoints. At 50K requests/sec, saving 2 page reads per request eliminates 100K random reads/second.

Interview Questions & Answers

Q1. Walk me through diagnosing a query that went from 100ms to 10 seconds overnight.

A: Step 1: Check `pg_stat_statements` for the query's recent performance change. Step 2: Run `EXPLAIN (ANALYZE, BUFFERS)` — look for estimated vs actual row discrepancy. Step 3: If estimates are wrong, run `ANALYZE` on the table and re-run `EXPLAIN`. Step 4: Check the execution plan for changes — was an index used before but not now? Step 5: Check if the table grew significantly (data growth changes cost calculations). Step 6: Check if an index was dropped or statistics became stale. Step 7: Look for `Sort Method: external` or `Batches > 1` indicating `work_mem` issues. Step 8: Check for lock waits via `pg_stat_activity`.

Q2. A query uses `WHERE date_trunc('month', created_at) = '2024-06-01'`. Why is it slow and how do you fix it?

A: `date_trunc('month', created_at)` is a function applied to the `created_at` column. An index on raw `created_at` won't be used because the index stores raw timestamps, not truncated ones. Fix: rewrite as a range query that the index can use: `WHERE created_at >= '2024-06-01' AND created_at < '2024-07-01'`. This is logically equivalent, uses the index, and is clear. Alternative if rewrite is impossible: create an expression index `CREATE INDEX ON events(date_trunc('month', created_at::timestamp))`.

Q3. What is the most impactful single change you can make to a slow query that scans a large table?

A: Add an appropriate index. For a `WHERE` condition that filters to a small fraction of rows (< 5%), adding a B-tree index on the `WHERE` column(s) typically reduces query time by 100-1000x. The index must be: (1) on the correct column(s), (2) with the right column order (equality before range), (3) created `CONCURRENTLY` to avoid table lock. After adding the index, run `ANALYZE` to update statistics and verify with `EXPLAIN ANALYZE`.

Cross-References

- [01 explain basics.md](#) — Reading `EXPLAIN` output
- [02 explain analyze.md](#) — `EXPLAIN ANALYZE` details
- [03 scan types.md](#) — Understanding scan types
- [04 join algorithms.md](#) — Join algorithm optimization
- [08 query rewriting.md](#) — Anti-pattern rewrites
- [09 slow query analysis.md](#) — Finding slow queries
- [../07 Indexes/](#) — All index types and design strategies